

Privacy-Preserving Asynchronous Federated Learning Framework in Distributed IoT

Xinru Yan¹, Yinbin Miao¹, *Member, IEEE*, Xinghua Li¹,
Kim-Kwang Raymond Choo², *Senior Member, IEEE*,
Xiangdong Meng, and Robert H. Deng³, *Fellow, IEEE*

Abstract—To solve the data island issue in the distributed Internet of Things (IoT) without privacy leakage, privacy-preserving federated learning (PPFL) has been extensively explored in both academic and industrial fields. However, existing PPFL solutions still suffer from a single point of failure and incur untrusted aggregation results caused by a malicious central server, and even cause a loss of model accuracy in an asynchronous setting. To solve these issues, we propose a privacy-preserving asynchronous federated learning scheme by using blockchain. Specifically, we use blockchain to address single points of failure and untrustworthy aggregation results, implement reliable model aggregation utilizing a practical byzantine fault-tolerant protocol in an asynchronous setting, and leverage differential privacy to improve system robustness. Formal security analysis and convergence analysis demonstrate that the proposed scheme is secure and robust, and extensive experiments demonstrate that our scheme can effectively ensure the accuracy of the system when compared with state-of-the-art schemes.

Index Terms—Asynchronous training, blockchain, differential privacy (DP), federated learning (FL).

Manuscript received 24 November 2022; revised 24 February 2023; accepted 24 March 2023. Date of publication 28 March 2023; date of current version 25 July 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 62072361; in part by the Key Research and Development Program of Shaanxi under Grant 2022GY-019; in part by the Shaanxi Fundamental Science Research Project for Mathematics and Physics under Grant 22JY019; in part by the National Natural Science Foundation of China under Grant 62125205; in part by the Fellowship of China Postdoctoral Science Foundation under Grant 2022T150507; in part by the Opening Project of Intelligent Policing Key Laboratory of Sichuan Province under Grant ZNJW2023KFMS002; and in part by the Henan Key Laboratory of Network Cryptography Technology & State Key Laboratory of Mathematical Engineering and Advanced Computing under Grant LNCT2020-A06. The work of Kim-Kwang Raymond Choo was supported by the Cloud Technology Endowed Professorship. (*Corresponding author: Yinbin Miao.*)

Xinru Yan and Yinbin Miao are with the School of Cyber Engineering, Xidian University, Xi'an 710071, China, also with the Henan Key Laboratory of Network Cryptography Technology, Zhengzhou 450001, China, and also with the State Key Laboratory of Mathematical Engineering and Advanced Computing, Zhengzhou 450001, China (e-mail: yanxinru25@163.com; ybmiao@xidian.edu.cn).

Xinghua Li is with the State Key Laboratory of Integrated Service Networks, School of Cyber Engineering, Xidian University, Xi'an 710071, China, and also with the Engineering Research Center of Big Data Security, Ministry of Education, Xi'an 710071, China (e-mail: xhli1@mail.xidian.edu.cn).

Kim-Kwang Raymond Choo is with the Department of Information Systems and Cyber Security, The University of Texas at San Antonio, San Antonio, TX 78249 USA (e-mail: raymond.choo@fulbrightmail.org).

Xiangdong Meng is with the Henan Key Laboratory of Network Cryptography Technology, Zhengzhou 450001, China, and also with the State Key Laboratory of Mathematical Engineering and Advanced Computing, Zhengzhou 450001, China (e-mail: xiangdong129@163.com).

Robert H. Deng is with the School of Information Systems, Singapore Management University, Singapore 178902 (e-mail: robertdeng@smu.edu.sg). Digital Object Identifier 10.1109/IIOT.2023.3262546

I. INTRODUCTION

IN THE distributed Internet of Things (IoT) environment, a large amount of data is generated. In order to mine the value of data, machine learning (ML) has been widely studied. Since a single IoT device has poor ML performance due to insufficient local data, federated learning (FL) is proposed to solve data islands and achieve efficient resource utilization. However, even though FL does not directly leak local data to the cloud server, the parameters are still subject to inference attacks. Therefore, privacy-preserving FL (PPFL) [1] has been extensively explored in IoT. However, existing PPFL solutions still face many issues in real-world applications.

The first issue is that traditional PPFL solutions still suffer from a single point of failure or untrusted aggregation [2] caused by a malicious central server. Specifically, once the central server is attacked, it may cause the system to go down and fail to operate normally. Even an attacker compromising the central server can falsify the data, ultimately resulting in untrusted aggregation results. The multiserver framework can be deployed in the system to address a single point of failure, but it cannot guarantee that the aggregation results are credible. Although various techniques are available to solve the verification problem, such as zero-knowledge proofs [3], signatures [4], and aggregate accumulators [5], these methods will incur high computation overheads on resource-limited clients. Now an available solution is to use blockchain [6], which can solve both issues with one stone. However, the traditional public network still leads to privacy leakage due to the involvement of nonclient, and even causes fault tolerance in asynchronous PPFL due to the use of consensus protocols, such as proof-of-work [7] and proof-of-stake [8]. To overcome the shortcoming of the public blockchain, the consortium blockchain composed of a small number of nodes can be deployed in PPFL without the involvement of untrusted nodes. Unfortunately, there are no asynchronous PPFLs implemented in the consortium blockchain network.

The second issue is how to achieve the proposed asynchronous PPFL. In a typical PPFL framework, multiple clients may suffer failures due to different computing powers [9]. The synchronous PPFL architecture needs to wait for delayed clients to achieve better results, otherwise, it will lead to a loss of model accuracy. For example, when some enterprises' devices are offline, it can be costly to spend a lot of time and computing power waiting for the devices

online. Timestamp-based FL schemes [10] can be exploited to address the asynchronous problem, but cannot provide privacy protection. Existing asynchronous PPFL schemes use secure multiparty computation (SMC) [11] or homomorphic encryption (HE) [12] mechanisms to protect privacy but incur high communication costs and computation costs, respectively. Thus, how to achieve an efficient asynchronous PPFL is still the problem we want to solve.

As mentioned above, we propose a privacy-preserving asynchronous FL framework in distributed IoT. Specifically, we use the blockchain to avoid single points of failure and guarantee the correctness of aggregation results. Furthermore, we use the practical Byzantine fault tolerant (PBFT) protocol [13] to guarantee the accuracy and reliability of asynchronous aggregation. Considering the lightweight blockchain architecture, we use differential privacy (DP) [14] to protect the local model and achieve efficient asynchronous aggregation. The main contributions of our work are as follows.

- 1) We propose a centerless DP-based PPFL with blockchain architecture. Specifically, we use the blockchain to prevent aggregators from malicious modifications or attempting to infer private information of clients during aggregation and utilize DP to prevent privacy breaches due to curious attempts.
- 2) We use asynchronous mechanisms to prevent system heterogeneity. The asynchronous training is accomplished by using PBFT to improve the trust in the system itself and the feasibility of practical applications.
- 3) Formal security analysis and convergence analysis prove that our scheme is secure and robust, and extensive experiments demonstrate that our scheme can effectively ensure the accuracy of the system when compared with state-of-the-art schemes.

The remainder of this article is organized as follows. Sections II and III describe the related work and techniques, respectively. In Section IV, we outline the system model, problem definition, threat model, and design goals of our scheme. Section V describes the details of our schemes. Then, we analyze the security and performance of the scheme in Sections VI and VII, respectively. Finally, we conclude this article and give a brief overview of future work.

II. RELATED WORK

McMahan et al. [15] proposed the classical synchronized Federated Averaging algorithms (FedAvg), which not only uses synchronization to schedule clients but also utilizes weighted averaging to update client gradients to form a global model. However, even if an FL does not directly leak local data to cloud servers, parameters are still subject to inference attacks. Therefore, researches on PPFL are very necessary. Fang and Qian [16] proposed a multiparty privacy-preserving ML framework, which uses HE encrypted gradients to protect the security of private data and uses a packing method to reduce transpose and rotation operations in matrix-vector multiplication. Furthermore, SMC techniques enable privacy-preserving model aggregation for an FL but incur high communication overhead and low scalability. To solve this problem, Kanagavelu et al. [17] developed

a two-stage mechanism, first electing a small committee and then providing model aggregation services for more participants through the committee. As each client's performance varies, the client may not complete local updates in time or even experience downtime, which eventually leads to delays. To eliminate the defect of the synchronous algorithm and prevent some situations of unpredictable clients, asynchronous training [18] has been extensively explored. For example, Zhou et al. [19] proposed a time-triggered FL algorithm over wireless networks, which uses the greedy search algorithm to solve the problem of user selection and bandwidth optimization. Two PPFL schemes, DeepPAR and DeepDPA, were proposed by Zhang et al. [20]. DeepPAR inherently preserves the secrecy of dynamic updates while protecting the input privacy of each participant. Meanwhile, DeepDPA can guarantee the backward secrecy of group participants in a lightweight manner. These schemes are all implemented under the centralized architecture. However, the stability of traditional centralized PPFL schemes cannot be guaranteed due to factors, such as the low-reliability [21] or biases [22] of the cloud service providers, which makes the servers vulnerable to poisoning attacks [23].

To solve the above problems, endowing an FL with a blockchain can ensure distributed data sharing without revealing privacy [24], and effectively optimize the security of PPFL. Lu et al. [25] designed a blockchain-authorized secure data-sharing PPFL structure, which reduces the risk of data leakage and protects privacy through distributed multiparty contribution data. Awan et al. [26] proposed a blockchain-based PPFL framework, which utilizes HE to perform gradient aggregation on private data to protect the model. Li et al. [1] designed a PPFL framework by using SMC, single-mask mechanism, and chained-communication mechanism to achieve practical privacy protection with low communication cost and without affecting the accuracy and convergence speed of the training model. To solve the synchronous problem, Liu et al. [27], respectively, proposed advanced FedBlock and FedAC models of distributed and an asynchronous FL frameworks to improve the robustness of the system. However, these schemes do not address the privacy issues of Blockchain-enabled FL. Lu et al. [28] proposed a hybrid blockchain architecture, which uses deep reinforcement learning for node selection and an asynchronous FL to improve the security and reliability of model parameters. Feng et al. [29] used a novel entropy weight approach to evaluate the participation level and proportion of local models trained in devices scheme. The energy consumption and local model update efficiency are balanced by adjusting the local training and communication delays, and optimizing the block generation rate.

However, the above solutions cannot ensure the accuracy and security of asynchronous PPFL in consortium blockchain. Therefore, we propose a privacy-preserving asynchronous FL framework in distributed IoT. The comparison of our scheme with the above schemes is shown in Table I.

III. PRELIMINARIES

In this section, we describe the related technologies involved in our scheme, including an FL, a blockchain, and a DP.

TABLE I
COMPARATIVE SUMMARY BETWEEN OUR
SCHEMES AND PREVIOUS SCHEMES

Schemes	Architecture	Category	Privacy mechanism
[15]	Centralized	synchronous	✗
[16]	Centralized	synchronous	HE
[17]	Centralized	synchronous	SMC
[18]	Centralized	asynchronous	✗
[19]	Centralized	asynchronous	✗
[20]	Centralized	asynchronous	HE
[25]	Decentralized	synchronous	DP
[26]	Decentralized	synchronous	HE
[1]	Decentralized	synchronous	SMC
[27]	Decentralized	asynchronous	✗
[28]	Decentralized	asynchronous	✗
[29]	Decentralized	asynchronous	✗
Our scheme	Decentralized	asynchronous	DP

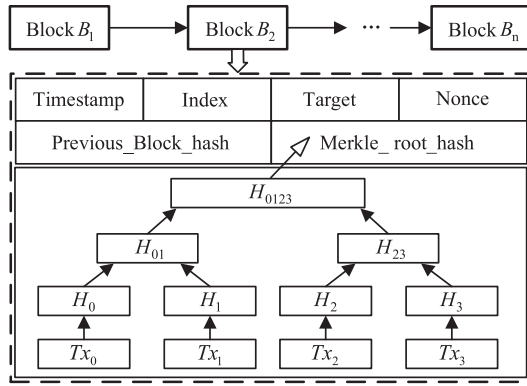


Fig. 1. Blockchain structure.

A. Federated Learning

Let $\mathcal{C} = \{C_1, C_2, \dots, C_n\}$ be n clients and $\mathcal{D} = \{D_1, D_2, \dots, D_n\}$ be corresponding data sets, the overall goal of FL is to obtain an optimal global model w^* . Given the global model w_t of the t th epoch, the client C_i ($i \in [1, n]$) updates its local model w_t^i by

$$w_t^i = w_t - \lambda * \nabla l(w_t; D_i) \quad (1)$$

where τ is the timestamp of C_i , λ is the learning rate, and $\nabla l(w_t; D_i)$ is the loss function of the training model. After that, C_i uploads w_t^i to the central server. Then, the central server receives the local models $\{w_t^1, w_t^2, \dots, w_t^n\}$ returned by $\mathcal{C}$, and aggregates them to update the global model w_{t+1} with a weighted average algorithm defined by

$$w_{t+1} = \frac{1}{n} \sum_{i=1}^n w_t^i. \quad (2)$$

B. Blockchain

Blockchain is a distributed shared ledger that stores all transactions in the network [6]. Fig. 1 shows the blockchain structure. For a block B_i with index i , Timestamp records the generation time of B_i . Target is negatively correlated with the

difficulty of mining and is calculated by

$$\text{Target} = \text{Target} \times \frac{t_a}{t_e} \quad (3)$$

where t_a is the actual time consumed to mine 2016 mines and t_e is the theoretical time consumed. Nonce is the random value to ensure that the hash value H_i of B_i 's block head matches Target by changing the Nonce value ($H_i \leq \text{Target}$). Previous_block_hash stores the hash H_{i-1} of the previous block head. Merkle_root_hash represents the transaction messages with the use of tree structure. The tree leaf nodes store the transactions $Tx = \{Tx_0, Tx_1, Tx_2, Tx_3\}$. It is worth noticing that Tx_j is the transaction with the serial number j , and is hashed to get H_j . After that, the neighboring nodes are hashed in turn until the root node H_{0123} is generated ($H_{0123} = \text{Hash}(H_{01} + H_{23})$).

C. Differential Privacy

DP [14] allows analysts to learn aggregate information about the data without access to the original private user data.

Definition 1 $[(\epsilon, \delta)\text{-DP}$ [30]]: For two adjacent data sets $\mathcal{D}_1$ and $\mathcal{D}_2$ that differ by only one record, given a randomized algorithm $\mathcal{K}$, the output result is $\mathcal{R} \subseteq \mathbb{R}$. If $\mathcal{K}$ and $\mathcal{R}$ satisfy the following:

$$\Pr[\mathcal{K}(\mathcal{D}_1) \in \mathcal{R}] \leq \exp(\epsilon) \Pr[\mathcal{K}(\mathcal{D}_2) \in \mathcal{R}] + \delta \quad (4)$$

the algorithm is said to conform to $(\epsilon, \delta)\text{-DP}$, where δ is the relaxation term and ϵ is the privacy budget.

Definition 2 (Laplace Mechanism [31]): Given data set $\mathcal{D}$ and query function $f(x)$, $\Delta f(x)$ is the sensitivity of $f(x)$, the random algorithm $\mathcal{K}$ needs to satisfy

$$\mathcal{K}(\mathcal{D}) = f(\mathcal{D}) + Y, Y \sim \text{Lap}\left(0, \frac{\Delta f(x)}{\epsilon}\right) \quad (5)$$

where ϵ is the privacy budget and Y is the randomly generated Laplacian noise.

IV. PROBLEM FORMULATION

In this section, we introduce the system model, problem definition, threat model, and design goals of our scheme.

A. System Model

We consider the consortium blockchain scenario. The system model of our scheme consists of three entities, namely, clients, blockchain, and primary node. The role of each entity is shown as follows.

- 1) **Clients**: Clients in the consortium blockchain need to both complete local model updates and participate in PBFT consensus to complete global model aggregation.
- 2) **Blockchain**: Blockchain uses an asynchronous mechanism to invoke the local model uploaded by clients to complete the aggregation.
- 3) **Primary Node**: The primary node acted by a client needs to trigger a new epoch of aggregation task at regular intervals with a timestamp and sends this aggregation result to all clients.

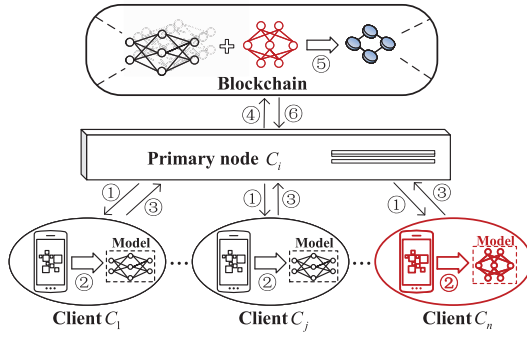


Fig. 2. System model of our basic scheme (In the t th epoch, C_i is the primary node and C_n drops out, at which point the red model of C_n stored in the blockchain is the expired model w_τ^t , where τ is the timestamp of C_n and $\tau \leq t$).

Our scheme is executed in the following steps. In the t th epoch global iteration, the primary node C_i ($i \in [1, n]$) sends the global model w_t to all clients $\mathcal{C} = \{C_1, C_2, \dots, C_n\}$ (step ①). The online client C_j ($j = \{1, 2, \dots, n\} \setminus \{i\}$) uses w_t to update the local model and adds noise to it in the t epoch of communication, resulting in w_τ^j , where τ is the timestamp of C_j (step ②). C_j upload parameter w_τ^j to the blockchain. Note that the red C_n in Fig. 2 is offline (step ③). C_i initiates the $(t+1)$ th epoch aggregation request to the blockchain network (step ④). In the aggregation, the expired models of the delayed clients (the red model pattern in Fig. 2) are aggregated with the $(t+1)$ th local models uploaded of the online clients to get the aggregation result w_{t+1} (step ⑤). Nodes in the network feed back w_{t+1} to C_i (step ⑥).

B. Problem Definition

The traditional PPFL undergoes a total of T epochs of updates, with n clients and a central server trained collaboratively. The overall goal is to form an optimal global model w^* . For the t th global iteration, the client set $\mathcal{C}$ uploads the local model set $\{w_\tau^1, w_\tau^2, \dots, w_\tau^n\}$, where τ is the timestamp of $\mathcal{C}$. Then, the central server aggregates these local models to update w_{t+1} . If the central server goes down, the aggregation stops. Even if the central server is compromised by an attacker, resulting in some models w_τ^i being tampered with ($w_\tau^i \neq w_t$) and eventually causing the aggregation result w^* to be untrusted.

In the case of synchronous training of multiple data sources, if m clients ($m < n$) are dropped due to network instability or poor computing powers (as shown in Fig. 2 with red C_n), only $n - m$ clients can communicate with the aggregator. We let $\mathcal{N}$ be the set of online clients, $\mathcal{J}$ be the set of uploaded local models of $\mathcal{N}$, and $\mathcal{Q}$ be the set of delayed clients. In this case, if the aggregator has to wait for $\mathcal{Q}$ to get online and then update w_{t+1} , it will incur higher latency and communication overhead. Conversely, if we give up waiting for $\mathcal{Q}$ and only aggregate $\mathcal{J}$, then, the accuracy of w_{t+1} accuracy will be drastically reduced, which leads to the incorrect w^* .

C. Threat Model

In this article, the client is considered to be “honest but curious” [32]. Under this assumption, the client will honestly perform local model updating and upload the right result to the blockchain but may be curious about other clients’ high-quality data, thereby posing a threat to the security of the data. In this article, the blockchain network in this article is considered to be fully trustworthy, which means that all clients recognize the results of global model aggregation performed in the blockchain network. The potential threats caused by clients and the central server are shown as follows.

- 1) *Leakage of Data Privacy*: Each client honestly runs the agreed protocol but is curious about the sensitive information contained by other clients. Therefore, the client may try to infer other clients’ private data, resulting in data leakage.
- 2) *Single Point of Failure*: Most of the existing FL schemes rely on the central server for aggregation. However, if the server is maliciously attacked, the aggregation result of the final training is prone to error, resulting in the failure of the FL training.
- 3) *Reduction of Model Accuracy*: In the actual deployment of an FL, the clients cannot timely feedback or even forced to appear offline phenomenon, as devices on each client have certain differences in computing, communication, and storage hardware and software. Whether waiting for the offline clients to go online or aggregating the online clients’ model will cause the loss of model accuracy.

D. Design Goals

By analyzing the above threat model and system model and combining the problems existing in the current research, we need to achieve the following functions and objectives.

- 1) *Privacy Protection*: Our scheme should implement DP, and add Laplacian noise to the updated local model to disturb the training parameters and ensure that the data privacy is not leaked.
- 2) *Asynchronous Training*: Our scheme should implement the asynchronous setting, trying to improve the training efficiency and prevent the waste of resources caused by system heterogeneity.
- 3) *Decentralized Architecture*: Our scheme should implement blockchain to build a decentralized FL system to prevent malicious cloud servers, thereby improving the security of training and ensuring the credibility of the system.
- 4) *Model Accuracy*: Our scheme should ensure model reliability, the obtained well-trained model is correctly aggregated from multiple local training models and not tampered with.

V. DESIGN OF OUR SCHEME

In this section, we describe how our scheme works. First, we give the technical overview of the scheme and then elaborate

TABLE II
NOTATIONS AND DESCRIPTIONS

Notation	Description	Notation	Description
B	Batch size	n	Total number of clients
$\mathcal{C}$	n clients set	ζ	Regularization weight
D_i	Datasets of C_i	w_t	t -th epoch global model
T	Global epoch	w_τ^i	τ -th epoch local model of C_i
Y	Laplacian noise	$z_{\tau,l}^i$	Data sampled from D_i
λ	Learning rate	$t - \tau$	Timestamp difference
s	sensitivity	α	Percent Byzantine nodes
ε	privacy budget	L	Local iterations
β	Hyper-parameter	$\mathcal{N}$	The set of online clients
$\mathcal{S}(\cdot)$	Function of β	$\mathcal{Q}$	The set of offline clients
$\ \cdot\ $	Default is l_2 -norms	$\mathcal{J}$	The set of normal local models of $\mathcal{N}$

on the implementation. In addition, Table II gives the notation definitions in this article.

A. Technical Overview

Traditional PPFL solutions suffer from a single point of failure and untrusted aggregation caused by malicious central servers. Li et al. [1] and Awan et al. [26], can address the above issues, but they incur high communication overheads and cannot solve the asynchronous problem.

We use the consortium blockchain to complete the global model updates, in which n nodes participate in the training as clients $\mathcal{C} = \{C_1, C_2, \dots, C_n\}$. The solution can tolerate up to f byzantine nodes that do not complete aggregation tasks properly ($f = [(n-1)3]$), but we need to guarantee that the size of $\mathcal{Q}$ is less than f ($|\mathcal{Q}| \leq f$). To avoid the dropout issue, we use the PBFT protocol to complete the aggregation. Specifically, the primary node C_i ($i \in [1, n]$) initiates the $(t+1)$ th aggregation request m_{t+1} with the timestamp action. After that, C_i computes and broadcasts the **pre-prepare** message $m_{t+1,i}^{pp}$ to the whole network. Then, each online node $C_j \in \mathcal{N}$ ($j \neq i$) receives and processes $m_{t+1,i}^{pp}$, and then broadcasts the **prepare** message $m_{t+1,j}^p$ to the blockchain network. C_j needs to ensure that it receives at least $2f+1$ legitimate messages $\{m_{t+1,1}^p, m_{t+1,2}^p, \dots, m_{t+1,2f+1}^p\}$, and then packages these messages before sending **commit** message $m_{t+1,j}^c$. Meanwhile, C_j starts to calculate m_{t+1} using an asynchronous mechanism after receiving at least $2f+1$ legitimate messages $\{m_{t+1,1}^c, m_{t+1,2}^c, \dots, m_{t+1,2f+1}^c\}$. Specifically, for the expiration model of the delayed client C_k ($C_k \in \mathcal{Q}$) stored in the blockchain, its t th epoch weight value β_t^k can be determined by the function $\mathcal{S}(\cdot)$. After that, C_j sends a **reply** message $m_{t+1,j}^r$ to C_i . C_i receives $f+1$ consistent messages $\{m_{t+1,1}^r, m_{t+1,2}^r, \dots, m_{t+1,f+1}^r\}$ to successfully obtain w_{t+1} . To eliminate the huge overhead in the network, DP makes it possible to protect the privacy security of lightweight blockchains. In the **local training** phase, $\mathcal{N}$ updates the set of local models $\mathcal{J}$ and uses the DP technology to add noise Y to it before uploading $\mathcal{J}$. Therefore, we propose a privacy-preserving asynchronous FL framework in distributed IoT.

Algorithm 1: Privacy-Preserving Asynchronous FL Framework in Distributed IoT

Input: $n, w_0, T, L, \mathcal{C}, \mathcal{N}, \mathcal{Q}, \mathcal{J}, \text{Time}$.
Output: w^* .

```

1  $C_i$  initialize  $w_0$ ;
2 for each  $t \in [0, T]$  do
3    $\mathcal{N} \leftarrow \emptyset, \mathcal{Q} \leftarrow \emptyset, \mathcal{J} \leftarrow \emptyset$ ;
4   Scheduler:
5    $C_i$  downloads  $(w_t, t)$  from Blockchain;
6    $C_i$  sends  $(w_t, t)$  to  $\mathcal{C}$ ;
7   while  $\text{time} \leq \text{Time}$  do
8     for  $C_j \in \mathcal{C}$  in parallel do
9       Local training in Algorithm 2:
10       $(w_\tau^j, \tau) \leftarrow \text{LocalUpdate}(w_t, t, L)$ ;
11       $C_j$  sends  $(w_\tau^j, \tau)$  to the blockchain;
12      Add  $C_j$  to online clients set  $\mathcal{N}$ ;
13      Add  $w_\tau^j$  to normal local model set  $\mathcal{J}$ ;
14    if  $|\mathcal{J}| == n$  then
15      break;
16   $\mathcal{Q} \leftarrow \mathcal{C} - \mathcal{N}$ ;
17  Model aggregation in Algorithm 3:
18   $(w_{t+1}, t+1) \leftarrow \text{Aggregation}(\mathcal{J}, \mathcal{N}, \mathcal{Q}, \beta, w_t, t)$ ;
19   $t \leftarrow t+1$ ;
20  $w^* \leftarrow w_t$ ;
21 return  $w^*$ .
```

B. Privacy-Preserving Asynchronous Federated Learning Framework in Distributed IoT

Based on the description of the program process, we divide the workflow of our scheme into three parts: 1) **scheduler**; 2) **local training**; and 3) **model aggregation**. The scheduler and model aggregation are performed asynchronously, which means that the threads on both sides are nonblocking. It is worth noticing that this method is very beneficial to the case of heterogeneous devices in practical applications. The algorithm of our scheme is shown in Algorithm 1. Initially, the primary node C_i in the blockchain initializes a global model w_0 and then obtains an optimized global model w^* after completing T epochs of aggregations.

Scheduler: The primary node C_i triggers the training task at regular intervals. C_i first fetches the t th epoch global model w_t from the blockchain, and then sends (w_t, t) to the client set $\mathcal{C}$ and starts the timing. When the time reaches **Time**, the model aggregation is still forced to start even if the set $\mathcal{Q}$ is not all online yet. Of course, if $\mathcal{C}$ is all completed with local model updates, then, the model aggregation starts (line 18). The **Time** value is related to the timestamp difference $t - \tau$.

Local Training: We describe the task of the local training in Algorithm 2. After the client C_j receives the global model and its corresponding timestamp (w_t, t) pushed by the blockchain through C_i (line 1), then C_j uses this SGD algorithm to update the local model w_τ^j with completing L iterations. C_j first randomly selects samples $z_{\tau,l}^j$ in the data set D_j (line 5), and then starts iterative training $w_{\tau,l}^j$, where $g(\cdot; \cdot)$ is the loss function

Algorithm 2: LocalUpdate()

Input: w_t, t, L .
Output: w_{τ}^j, τ .

- 1 Receive (w_t, t) from C_i ; //Scheduler ;
- 2 $\tau \leftarrow t, w_{\tau,0}^j \leftarrow w_t$;
- 3 *Update local model:*
- 4 **for** local iteration $l \in [1, L]$ **do**
- 5 Randomly sample $z_{\tau,l}^j \sim D_j$;
- 6 $w_{\tau,l}^j \leftarrow w_{\tau,l-1}^j - \lambda * \nabla g(w_{\tau,l-1}^j; z_{\tau,l}^j)$;
- 7 *DP protection:*
- 8 $w_{\tau+1}^j \leftarrow w_{\tau}^j + Y$; // See the Eq. 7;
- 9 $\tau \leftarrow \tau + 1$;
- 10 **return** (w_{τ}^j, τ) .

(line 6). To speed up the convergence and ensure that the local model is closer to the global model, we define the loss function by

$$g(w_{\tau}^j; z_{\tau}^j) = f(w_{\tau}^j; z_{\tau}^j) + \frac{\zeta}{2} \|w_{\tau}^j - w_t\|^2 \quad (6)$$

where $f(\cdot; \cdot)$ is the local function and ζ is regularization weight [10]. To avoid user privacy leakage caused by reverse inferences of curious clients, DP can be used to further protect model data by

$$Y \sim \text{Lap}\left(0, \frac{s}{\epsilon}\right), w_{\tau+1}^j = w_{\tau}^j + Y \quad (7)$$

where s is sensitivity, ϵ is privacy budget, and Y is randomly generated Laplacian noise (line 8). Finally, the model w_{τ}^j with noise and its timestamp τ are pushed to the blockchain.

Model Aggregation: Model aggregation is performed by the blockchain network after completing local training for the t th epoch, which is illustrated in Algorithm 3. Specifically, the primary node C_i initiates $(t+1)$ th epoch aggregation request m_{t+1} under the action of the fixed timestamp Time, where o, t , and σ_i denote operation, timestamp and the signature of C_i , respectively, (line 1). Then, C_i starts the PBFT consensus, which consists of three phases, including Pre-prepare, Prepare, and Commit. When starting the Pre-prepare phase, C_i calculates and sends a pre-prepare message $m_{t+1,i}^{pp}$ to the network, where v, n , and d denote view, series number, and the digest for m_{t+1} , respectively, (lines 2–4). After that, the Prepare phase starts and the other backup nodes need to check the validity of the received message $m_{t+1,i}^{pp}$. Specifically, C_j needs to ensure that σ_i and d are correct and to judge whether the current view is v , and then checks if a message with the same v and n but different d received. If all pass, it means the message is legal. After that, the node needs to change its status to pre-prepared, then calculates and broadcasts the prepare message $m_{t+1,j}^p$ to the whole network, where j is the ID of online backup node (lines 6–10). Finally, it comes to the Commit phase, C_j first needs to judge whether the received prepare message from other clients is legal. The verification method is similar to that in the Prepare phase, which requires inspection of n, d, v , and signature. Different from the above checks, C_j needs to

make sure that it gets at least $2f + 1$ such legitimate messages $\{m_{t+1,1}^p, m_{t+1,2}^p, \dots, m_{t+1,2f+1}^p\}$. Then, it calculates and broadcasts the commit message $m_{t+1,j}^c$, and changes its state to prepared (lines 11–15). After obtaining sufficient legitimate commit messages $\{m_{t+1,1}^c, m_{t+1,2}^c, \dots, m_{t+1,2f+1}^c\}$, C_j then changes its state to committed. Similarly, the inspection contents of n, d, v , and signatures are similar to that of the Commit phase (lines 16 and 17). At this time, the node needs to perform the m_{t+1} , which is the $(t+1)$ th epoch global model aggregation. For the normal local model w_{τ}^k ($k \in \mathcal{N}$) uploaded by the online client, the weight β_t^k is set to 1 (line 19). For the delayed local model w_{τ}^k ($k \in \mathcal{Q}$) of the disconnected client, the function $\mathcal{S}(\cdot)$ can be used to determine the weight β_t^k by

$$\mathcal{S}(t - \tau) = \frac{1}{2^{(t-\tau)}} \quad (8)$$

where t is the current global epoch and τ is the timestamp used by the delayed client (line 21). When the timestamp difference of $t - \tau$ is large, the corresponding value of β_t^k can be reduced by $\mathcal{S}(\cdot)$, thereby reducing the error caused by the lag. Then, C_j accumulates the weight values of all local models to get β_t (line 22). And, the global model w_{t+1} of $(t+1)$ th epoch is aggregated by (9) (line 23)

$$w_{t+1} = \sum_{k=1}^n \frac{\beta_t^k}{\beta_t} * w_{\tau}^k. \quad (9)$$

After completing model aggregation, the node C_j sends a reply message $m_{t+1,j}^R$ to C_i , where r is the processed global model result w_{t+1} (line 24). C_i completes the aggregation task for the $(t+1)$ th epoch, once it has received $f+1$ consistent reply messages $\{m_{t+1,1}^R, m_{t+1,2}^R, \dots, m_{t+1,f+1}^R\}$ (line 25).

VI. SECURITY ANALYSIS

In this section, we introduce the convergence analysis and security analysis of our scheme. In the convergence analysis, it can be concluded that the scheme can successfully converge to the critical point after completing T epochs of global update. The security analysis is described from three aspects: 1) privacy; 2) availability; and 3) credibility.

A. Convergence Analysis

Definition 3 (ι -Smooth): Let the smoothness parameter $\iota > 0$, the function f is ι -smooth, and if for $\forall x, y$

$$f(y) - f(x) - \langle \nabla f(x), y - x \rangle \leq \frac{\iota}{2} \|y - x\|^2. \quad (10)$$

Definition 4 (μ -Strongly Convex): Let constant $\mu > 0$, the function f is μ -strongly convex, and if for $\forall x, y$

$$f(y) - f(x) - \langle \nabla f(x), y - x \rangle \geq \frac{\mu}{2} \|y - x\|^2. \quad (11)$$

Theorem 1: Assume that the function $\mathcal{F}$ is ι -smooth and μ -strongly convex, the bound of the timestamp difference is $\mathcal{A} = \max(t - \tau)$, and the imbalance ratio of local training is $\eta = (L_1/L_0)$, where L_0 and L_1 are, respectively, the minimum and the maximum numbers of local training iterations. For $\forall w \in \mathbb{R}^d$, we have $\mathcal{V}_1 \geq \|\nabla f(w_{\tau}^i; z_{\tau}^i)\|^2$, $\mathcal{V}_2 \geq \|\nabla g(w_{\tau}^i; z_{\tau}^i)\|^2$. Let $\varphi > 0$ and $\zeta > \mu$, ($\zeta^2 -$

Algorithm 3: Aggregation()

Input: $\mathcal{J}, \mathcal{N}, \mathcal{Q}, \beta, w_t, t$.
Output: $w_{t+1}, t + 1$.

- 1 C_i initiates the request $m_{t+1} = \langle \text{request}, o, t \rangle_{\sigma_i}$;
- 2 **Pre-prepare phase:**
- 3 C_i calculates pre-prepare message
 $m_{t+1,i}^{pp} = \langle \langle \text{pre-prepare}, v, n, d \rangle_{\sigma_i}, m_{t+1} \rangle$;
- 4 C_i broadcasts the $m_{t+1,i}^{pp}$ to the whole network;
- 5 **for** $C_j \in \mathcal{N}$ **in parallel do**
- 6 **Prepare phase:**
- 7 Receive and verify the $m_{t+1,i}^{pp}$;
- 8 Change state to pre-prepared;
- 9 Calculate prepare message
 $m_{t+1,j}^p = \langle \text{prepare}, v, n, d, j \rangle_{\sigma_j}$;
- 10 Broadcast the $m_{t+1,j}^p$ to the whole network;
- 11 **Commit phase:**
- 12 Receive at least $2f + 1$ valid $m_{t+1,i}^p$;
- 13 Change state to prepared;
- 14 Calculate commit message
 $m_{t+1,j}^c = \langle \text{commit}, v, n, d, j \rangle_{\sigma_j}$;
- 15 Broadcast the $m_{t+1,j}^c$ to the whole network;
- 16 Receive at least $2f + 1$ valid $m_{t+1,i}^c$;
- 17 Change state to committed;
- 18 **for** $k \in \mathcal{J}$ **do**
- 19 $\beta_t^k \leftarrow 1$; //Set the weight of $\mathcal{N}$;
- 20 **for** $k \in \mathcal{Q}$ **do**
- 21 $\beta_t^k \leftarrow \mathcal{S}(t - \tau)$; //Set the weight of $\mathcal{Q}$;
- 22 $\beta_t \leftarrow \sum_{k=1}^n \beta_t^k$;
- 23 $w_{t+1} \leftarrow \sum_{k=1}^n (\beta_t^k / \beta_t * w_t^k)$; // Aggregation;
- 24 Send reply message $m_{t+1,j}^R = \langle \text{reply}, v, t, i, j, r \rangle_{\sigma_j}$ to C_i ;
- 25 $w_{t+1} \leftarrow C_i$ receives at least $f + 1$ valid $m_{t+1,i}^R$;
- 26 **return** $(w_{t+1}, t + 1)$.

$[\zeta/2])\|w_{\tau,l-1} - w_\tau\|^2 \geq (1 + 2\zeta + \varphi)\mathcal{V}_2$ [10]. After completing T epochs of aggregations, our scheme converges to a critical point w^*

$$\begin{aligned}
& \min_{i=0}^{T-1} \mathbb{E} \|\nabla \mathcal{F}(w^*)\|^2 \\
& \leq \frac{\mathbb{E}[\mathcal{F}(w_0) - \mathcal{F}(w^*)]}{\beta\lambda\varphi TL_0} + \mathcal{O}\left(\frac{\lambda\eta L_1^2}{\varphi}\right) + \mathcal{O}\left(\frac{\beta\eta\mathcal{A}}{\varphi}\right) \\
& \quad + \mathcal{O}\left(\frac{\beta^2\lambda\eta\mathcal{A}^2L_1}{\varphi}\right) + \mathcal{O}\left(\frac{\lambda\eta\mathcal{A}^2L_1}{\varphi}\right). \tag{12}
\end{aligned}$$

Proof: We assume that in the t th epoch, the client C_i uploads the local model and its timestamp (w_τ^i, τ) , L is the iteration number of C_i 's local training ($L_0 \leq L \leq L_1$). Let $\mathcal{A} = \max(t - \tau)$ and ignore i for convenience, then we can get

$$\begin{aligned}
& \|w_\tau - w_{t-1}\| \\
& \leq \|(w_\tau - w_{\tau+1}) + \dots + (w_{t-1} - w_{t-1})\| \\
& \leq \beta\lambda\mathcal{A}L_1\mathcal{O}(\sqrt{\mathcal{V}_2}). \tag{13}
\end{aligned}$$

According to Definition 4, we have

$$\begin{aligned}
& \mathcal{F}(w_\tau) - \mathcal{F}(w_{t-1}) \\
& \leq \|\nabla \mathcal{F}(w_{t-1})\| \|w_\tau - w_{t-1}\| + \frac{\iota}{2} \|w_\tau - w_{t-1}\|^2 \\
& \leq \beta\lambda\mathcal{A}L_1\mathcal{O}(\sqrt{\mathcal{V}_1\mathcal{V}_2}) + \beta^2\lambda^2\mathcal{A}^2L_1^2\mathcal{O}(\mathcal{V}_2). \tag{14}
\end{aligned}$$

Let $\forall l \in [L]$, then we can get

$$\begin{aligned}
& \mathbb{E}[\mathcal{F}(w_{\tau,l}) - \mathcal{F}(w^*)] \\
& \leq \mathcal{G}(w_{\tau,l-1}) - \mathcal{F}(w^*) + \frac{\iota\lambda^2}{2} \mathbb{E}[\|\nabla g(w_{\tau,l-1})\|^2] \\
& \quad - \lambda \mathbb{E}[\langle \nabla \mathcal{G}(w_{\tau,l-1}), \nabla g(w_{\tau,l-1}) \rangle] \\
& \leq \mathcal{F}(w_{\tau,l-1}) - \mathcal{F}(w^*) + \frac{\iota + \zeta L_1^2}{2} \lambda^2 \mathcal{V}_2 - \lambda\varphi \|\nabla \mathcal{F}(w_{\tau,l-1})\|^2 \\
& \leq \mathcal{F}(w_{\tau,l-1}) - \mathcal{F}(w^*) + \lambda^2 \mathcal{O}(\zeta L_1^2 \mathcal{V}_2) - \lambda\varphi \|\nabla \mathcal{F}(w_{\tau,l-1})\|^2 \\
& \leq \lambda^2 \mathcal{O}(\zeta L_1^3 \mathcal{V}_2) - \lambda\varphi \sum_{l=0}^{L-1} \mathbb{E} \|\nabla \mathcal{F}(w_{\tau,l})\|^2. \tag{15}
\end{aligned}$$

Then, we have

$$\begin{aligned}
& \mathbb{E}[\mathcal{F}(w_t) - \mathcal{F}(w_{t-1})] \\
& \leq \mathbb{E}[\beta\mathcal{G}(w_{\tau,L}) + (1 - \beta)\mathcal{G}(w_{t-1}) - \mathcal{F}(w_{t-1})] \\
& \leq \mathbb{E}\left[\beta(\mathcal{F}(w_{\tau,L}) - \mathcal{F}(w_{t-1})) + \frac{\beta\zeta}{2} \|w_{\tau,L} - w_{t-1}\|^2\right] \\
& \leq \beta\mathbb{E}[\mathcal{F}(w_{\tau,L}) - \mathcal{F}(w_{t-1})] + \beta\zeta\lambda^2\mathcal{A}^2L_1^2\mathcal{O}(\mathcal{V}_2) \\
& \leq -\beta\lambda\varphi \sum_{l=0}^{L-1} \mathbb{E} \|\nabla \mathcal{F}(w_{\tau,l})\|^2 + \beta\lambda^2\zeta L_1^3\mathcal{O}(\mathcal{V}_2) \\
& \quad + \beta\lambda^2\mathcal{A}^2L_1^2\mathcal{O}(\mathcal{V}_2) + \beta^3\lambda^2\mathcal{A}^2L_1^3\mathcal{O}(\mathcal{V}_2) \\
& \quad + \beta^2\lambda\mathcal{A}L_1\mathcal{O}(\sqrt{\mathcal{V}_1\mathcal{V}_2}). \tag{16}
\end{aligned}$$

Let L_t be the number of iterations for local model updates in the t th iteration, then we can get

$$\begin{aligned}
& \sum_{l=0}^{L_t-1} \mathbb{E} \|\nabla \mathcal{F}(w_{\tau,l})\|^2 \\
& \leq \frac{\mathbb{E}[\mathcal{F}(w_{t-1}) - \mathcal{F}(w_t)]}{\beta\lambda\varphi} + \frac{\lambda L_t^3}{\varphi} \mathcal{O}(\mathcal{V}_2) \\
& \quad + \frac{(\beta^2 + 1)\lambda\mathcal{A}^2L_1^2}{\varphi} \mathcal{O}(\mathcal{V}_2) + \frac{\beta\mathcal{A}L_1}{\varphi} \mathcal{O}(\sqrt{\mathcal{V}_1\mathcal{V}_2}). \tag{17}
\end{aligned}$$

After completing T epochs of aggregations, we have

$$\begin{aligned}
& \min_{t=0}^{T-1} \mathbb{E}[\|\nabla \mathcal{F}(w_t)\|^2] \\
& \leq \frac{1}{\sum_{t=1}^T L_t} \sum_{t=1}^T \sum_{l=0}^{L_t-1} \|\nabla \mathcal{F}(w_{\tau,l})\|^2 \\
& \leq \frac{\mathbb{E}[\mathcal{F}(w_0) - \mathcal{F}(w_T)]}{\beta\lambda\varphi TL_0} + \frac{(\beta^2 + 1)\lambda\mathcal{A}^2TL_1^2}{\varphi TL_0} \mathcal{O}(\mathcal{V}_2) \\
& \quad + \frac{\lambda TL_1^3}{\varphi TL_0} \mathcal{O}(\mathcal{V}_2) + \frac{\beta\mathcal{A}TL_1}{\varphi TL_0} \mathcal{O}(\sqrt{\mathcal{V}_1\mathcal{V}_2}) \\
& \leq \frac{\mathbb{E}[\mathcal{F}(w_0) - \mathcal{F}(w^*)]}{\beta\lambda\varphi TL_0} + \mathcal{O}\left(\frac{\lambda\eta L_1^2}{\varphi}\right) + \mathcal{O}\left(\frac{\beta\eta\mathcal{A}}{\varphi}\right) \\
& \quad + \mathcal{O}\left(\frac{\beta^2\lambda\eta\mathcal{A}^2L_1}{\varphi}\right) + \mathcal{O}\left(\frac{\lambda\eta\mathcal{A}^2L_1}{\varphi}\right). \tag{18}
\end{aligned}$$

■

B. Security Analysis

1) *Privacy*: In FL, each client is independent of the other and may be curious about the parameters during learning interactions, so as to reverse reasoning to obtain the privacy information of other clients. Therefore, to protect the training parameters, this scheme adds randomly generated Laplacian noise to the local model of each client by using DP. Laplace is a continuous distribution and is shown by

$$f(w|\mu, b) = \frac{1}{2b} \exp\left(-\frac{|w - \mu|}{b}\right) \quad (19)$$

where μ is the position parameter, b is the scale parameter, its mathematical expectation is μ , and variance is $2b^2$. The concrete visible (5) based on Laplacian DP, where $f(D)$ represents the query function, $K(D)$ is the result of confusion after adding Laplacian noise to $f(D)$, thus, Y satisfies ε -DP.

Proof: We use Laplace to achieve DP

$$\begin{aligned} & \frac{\Pr[K(D_1) = t]}{\Pr[K(D_2) = t]} \\ &= \frac{\prod_i^k (\varepsilon/2\Delta f) \exp(-\varepsilon|t_i - f(D_1)_i|/\Delta f)}{\prod_i^k (\varepsilon/2\Delta f) \exp(-\varepsilon|t_i - f(D_2)_i|/\Delta f)} \\ &= \exp\left(\frac{\varepsilon\left[-\sum_i^k |t_i - f(D_1)_i| - \sum_i^k |t_i - f(D_2)_i|\right]}{\Delta f}\right) \\ &\leq \exp\left(\frac{\varepsilon\|f(D_1) - f(D_2)\|_1}{\Delta f}\right) \\ &= \exp(\varepsilon). \end{aligned} \quad (20)$$

2) *Availability*: FL is usually trained by synchronous scheduling, that is, the server can update the global model only after all clients complete local updates. However, in actual deployment, the uneven performance of each client needs to be balanced, which leads to a waste of computing resources and time and low learning efficiency. In addition, the security and accuracy of the system will be greatly reduced if a client fails to return results in a timely manner. Therefore, this scheme uses an asynchronous scheduling mode to improve training efficiency while ensuring system security and model accuracy.

3) *Credibility*: The traditional FL solutions rely on a cloud server, but the stability of this server cannot be guaranteed and its credibility is low, which forces the global model to be limited, resulting in the weak credibility of the system. Therefore, this scheme proposes to use blockchain to replace traditional cloud servers and integrate untrusted clients to cooperate and train together, which aims to form a safe and reliable decentralized ledger.

VII. EXPERIMENTS ANALYSIS

In this section, we use MNIST, Fashion-MNIST, and Cifar-10 data sets to benchmark and analyze our scheme.

A. Experimental Setting

The test benchmark of the scheme is MNIST [9], Fashion-MNIST [33], and Cifar-10 data sets. Among them, the MNIST

TABLE III
PARAMETER DESCRIPTIONS

Param	L	B	n	ζ
Values	5	10	{10,20}	{1,0.1,0.01,0.001,0.0001}
Param	T	λ	$\max(t - \tau)$	α
Values	100	0.01	{0.4,0.8}	{0.1,0.2,0.3}

TABLE IV
UNDER THE THREE DATA SETS, THE INFLUENCE OF DIFFERENT ζ VALUES ON THE TEST ACCURACY

Datasets	ζ Values	1	0.1	0.01	0.001	0.0001
MNIST	FedProx	97.86%	97.95%	97.89%	97.82%	97.80%
	Our Scheme	97.81%	97.70%	97.85%	97.74%	97.81%
Fashion-Mnsit	FedProx	89.04%	89.08%	89.12%	89.03%	88.54%
	Our Scheme	88.17%	88.36%	88.67%	88.27%	88.30%
Cifar-10	FedProx	70.54%	70.27%	70.55%	70.12%	69.89%
	Our Scheme	68.19%	70.25%	69.61%	70.66%	68.45%

data set is widely used in the original FL [15] assessment and contains ten categories of handwritten images, while the fashion-MNIST data set contains ten categories of clothing images. Both data sets contain training 60 000 training samples and 10 000 testing samples. Cifar-10 data set contains ten categories of RGB color images, and the data set contains 50 000 training samples and 10 000 testing samples.

In each epoch of training, we choose all clients to participate in the training, but there may be someone clients offline as Byzantine nodes. Our experimental parameters are shown in Table III, where α represents the percentage of Byzantine nodes in total nodes at each training epoch, denoted by BN in Figs. 3–5. This means that at most $\alpha \times n$ clients are Byzantine nodes in each training epoch. Second, we simulate asynchronous scenarios in real applications by randomly determining the timestamp value $t - \tau$.

B. Comparative Analysis

Based on MNIST, Fashion-MNIST, and Cifar-10 data sets, we set the classical synchronous optimized FL algorithms, such as FedAvg [15], the Fedavg-based privacy protection and offline situation DropPPFL, and the FedProx [34] as comparison schemes, to evaluate the accuracy of our scheme.

1) *MNIST*: In Fig. 3(a), we measure the effect of different ζ values {1, 0.1, 0.01, 0.001, 0.0001} on the results of our scheme ($n = 20, t - \tau \leq 4, \alpha = 0.2$), and combined with Table IV, it can be found from the results produced by the nonoptimal ζ value are within the tolerable range, which the difference is less than 0.2%. Therefore, even if the optimal ζ value is not found in practice, our scheme can still achieve satisfactory results. In Fig. 3(b), we compare FedAvg, FedProx, and DropPPFL with our Scheme. It can be intuitively concluded that offline clients not only slow down the convergence speed and deteriorate the robustness, but also reduce the accuracy of the model by nearly 10%. In the case of $t - \tau \leq 4$, Fig. 3(c) and (d) tests the influence of different α values {0.1, 0.2, 0.3} on the accuracy of our scheme and FedProx when $n = 10$ and $n = 20$, respectively. We can find that

TABLE V
UNDER THE THREE DATA SETS, THE IMPACT OF DIFFERENT NUMBERS OF CLIENTS, DIFFERENT PERCENT
OF BYZANTINE NODES AND DIFFERENT TIMESTAMP DIFFERENCES ON TEST ACCURACY

Datasets	Values	$\max(t - \tau) = 4$				$\max(t - \tau) = 8$			
		$n = 10$		$n = 20$		$n = 10$		$n = 20$	
		FedProx	Our Scheme	FedProx	Our Scheme	FedProx	Our Scheme	FedProx	Our Scheme
Mnsit	$\alpha = 0.1$	97.90%	97.95%	97.86%	97.85%	97.86%	97.95%	97.98%	97.89%
	$\alpha = 0.2$	97.96%	98.06%	97.88%	97.79%	97.97%	97.90%	97.96%	97.87%
	$\alpha = 0.3$	97.82%	97.59%	97.80%	97.77%	97.86%	97.81%	97.84%	97.59%
Fashion-Mnsit	$\alpha = 0.1$	88.62%	88.71%	88.78%	88.64%	88.68%	88.54%	88.60%	88.31%
	$\alpha = 0.2$	88.73%	88.46%	88.60%	88.69%	88.80%	88.37%	88.71%	88.33%
	$\alpha = 0.3$	88.64%	87.79%	88.65%	88.32%	88.51%	87.86%	88.70%	87.96%
Cifar-10	$\alpha = 0.1$	70.40%	70.82%	70.08%	70.33%	69.95%	70.06%	69.76%	70.41%
	$\alpha = 0.2$	69.69%	67.68%	70.10%	68.69%	69.48%	66.03%	69.39%	68.54%
	$\alpha = 0.3$	69.30%	66.50%	69.61%	68.08%	68.24%	67.34%	69.92%	67.72%

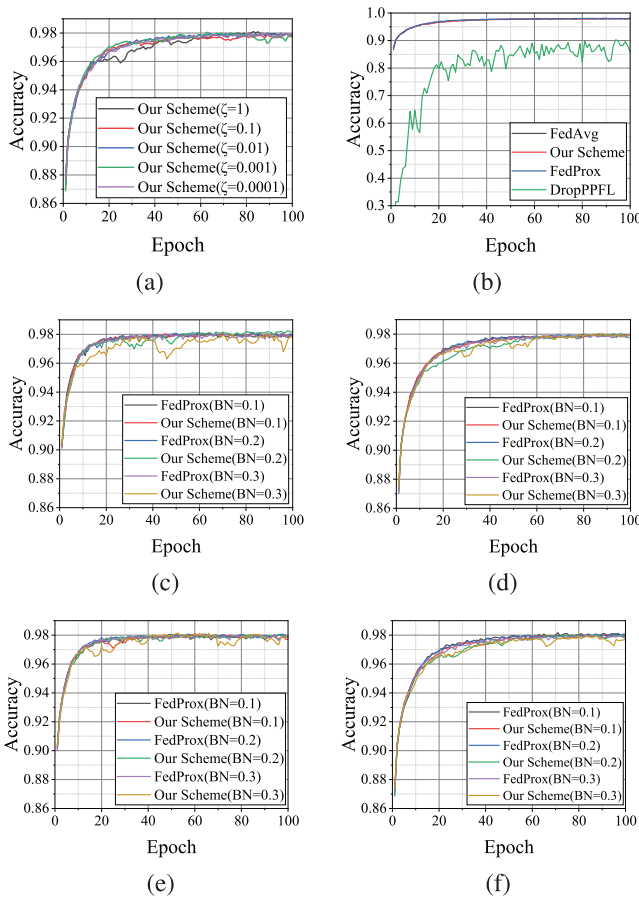


Fig. 3. MNIST test results. (a) $n = 20, t - \tau \leq 4$, (b) $n = 20, t - \tau \leq 4$, (c) $t - \tau \leq 4, n = 10$. (d) $t - \tau \leq 4, n = 20$. (e) $t - \tau \leq 8, n = 10$. (f) $t - \tau \leq 8, n = 20$.

with the increase of α , the convergence speed of our scheme slows down, and the range of line jitter becomes larger, but the accuracy loss is within the allowable range. As the number of clients increases, the number of epochs required for the schemes to reach convergence increases by 40, but it does not affect the final test accuracy of the scheme. Similarly, in the case of $t - \tau \leq 8$, the effects of different numbers of clients and different α on our scheme and FedProx are tested again, as shown in Fig. 3(e) and (f). We give the test accuracy

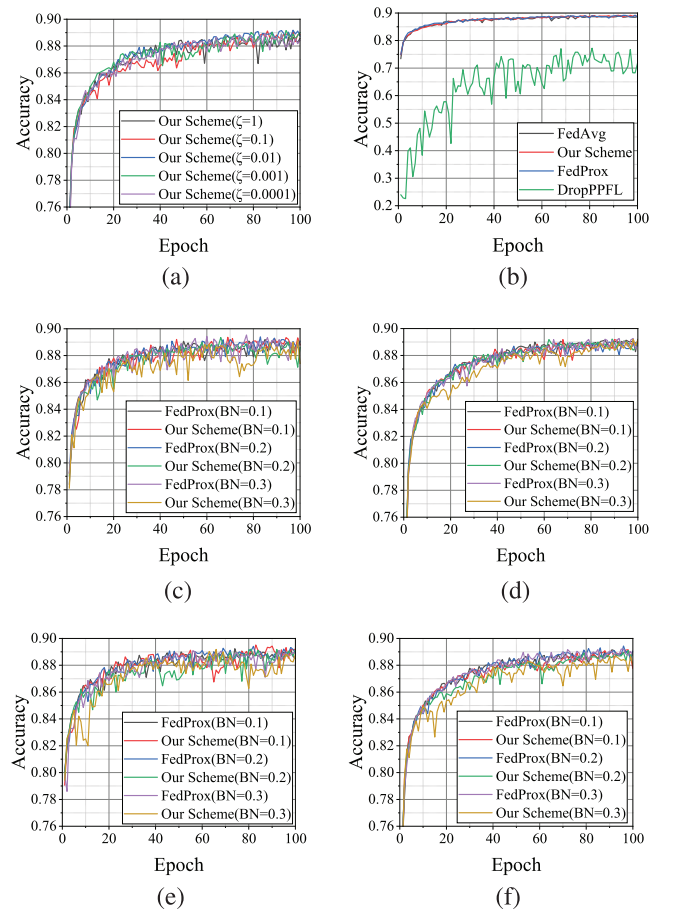


Fig. 4. FMNIST test results. (a) $n = 20, t - \tau \leq 4$, (b) $n = 20, t - \tau \leq 4$, (c) $t - \tau \leq 4, n = 10$. (d) $t - \tau \leq 4, n = 20$. (e) $t - \tau \leq 8, n = 10$. (f) $t - \tau \leq 8, n = 20$.

of Fig. 3(c)–(f) in Table V, from which we can find that the accuracy of the scheme has a little impact with the increase of the time stamp difference, and even better than that under the same parameter configuration in some tests $t - \tau \leq 4$ case (such as $\alpha = 0.1$).

2) *Fashion-MNIST*: In Fig. 4(a), the effects of different ζ values $\{1, 0.1, 0.01, 0.001, 0.0001\}$ on the accuracy of FedProx and our scheme are mainly shown ($n = 20, t - \tau \leq 4, \alpha = 0.2$). Also, combining Table IV shows that we do not

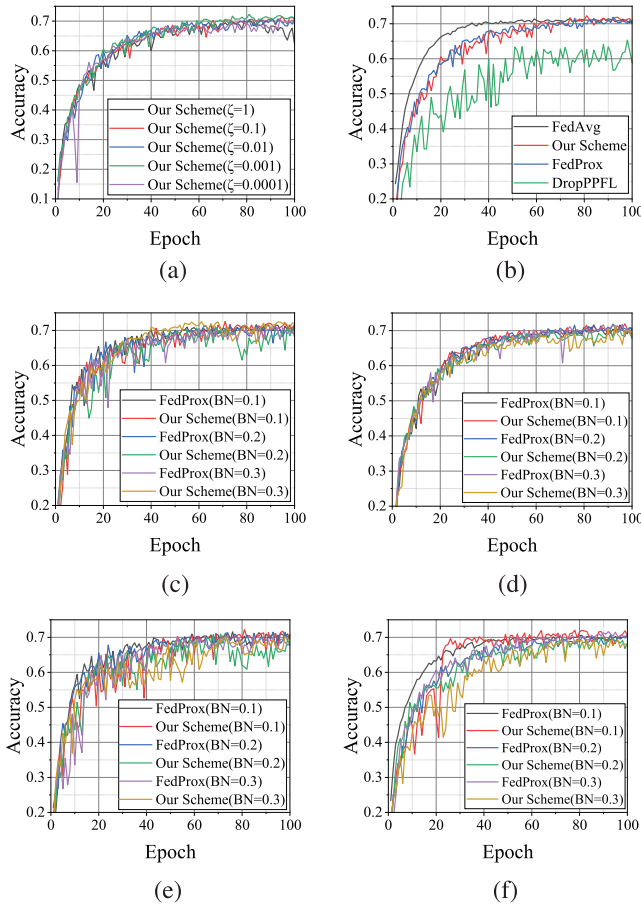


Fig. 5. Cifar-10 test results. (a) $n = 20, t - \tau \leq 4$, (b) $n = 20, t - \tau \leq 4$, (c) $t - \tau \leq 4, n = 10$. (d) $t - \tau \leq 4, n = 20$. (e) $t - \tau \leq 8, n = 10$. (f) $t - \tau \leq 8, n = 20$.

have to struggle with the optimal value of ζ . In Fig. 4(b), it can be found that the accuracy of the DropPPFL is reduced by nearly 15% compared with our scheme. In the case of $t - \tau \leq 4$ and $t - \tau \leq 8$, Fig. 4(c)–(f), respectively, tested our scheme and FedProx with different α values $\{0.1, 0.2, 0.3\}$ under the number of clients 10 and 20. The effect on protocol accuracy and test accuracies are given in Table V. We can find that with the increase of α , the line jitter of the scheme becomes larger but it does not affect the final test accuracy of the scheme. However, in the case of $n = 20$, the stability of the scheme will be better than that of $n = 10$, that is, when the percent of Byzantine nodes increases, the larger n is, the smaller the impact will be.

3) *Cifar-10*: We test Cifar-10 similarly to the above two data sets. In Fig. 5(a) and Table IV, the effects of different ζ values $\{1, 0.1, 0.01, 0.001, 0.0001\}$ on the accuracy of FedProx and our scheme are mainly shown ($n = 20, t - \tau \leq 4, \alpha = 0.2$). It can be found that FedProx has the highest accuracy when $\zeta = 0.01$ and our scheme has the highest accuracy when $\zeta = 0.001$. Also, we do not have to worry about the optimal value of ζ . In Fig. 5(b), it can be found that dropped clients slow down the convergence speed of all schemes. In addition, the accuracy of the DropPPFL model is reduced by nearly 8% compared with our scheme. In the case of $t - \tau \leq 4$ and $t - \tau \leq 8$, Fig. 5(c)–(f) tests the influence of different

α values $\{0.1, 0.2, 0.3\}$ on the accuracy of our scheme and FedProx when $n = 10$ and $n = 20$, respectively. And, the test accuracy is given in Table V. Similar to the FMNIST, Cifar-10 can cope well with the problem of dropped clients when the number of clients increases. And, it can be found that our scheme is better than $t - \tau \leq 8$ when dealing with $t - \tau \leq 4$.

VIII. CONCLUSION

In this article, we use blockchain instead of a cloud server to design an asynchronous PPFL without disclosing the privacy of client data. We evaluate our scheme based on three different data sets. The results show that our scheme is able to address the lack of precision and low security. However, the efficiency of consensus urgently needs to be improved. In future work, we will also consider how to reduce the time consumption of PPFL in distributed IoT, in addition to improving the accuracy of training results.

REFERENCES

- [1] Y. Li, Y. Zhou, A. Jolfaei, D. Yu, G. Xu, and X. Zheng, "Privacy-preserving federated learning framework based on chained secure multi-party computing," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6178–6186, Apr. 2021.
- [2] Y. Qu et al., "Decentralized privacy using blockchain-enabled federated learning in fog computing," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 5171–5183, Jun. 2020.
- [3] J. Zhang, Z. Fang, Y. Zhang, and D. Song, "Zero knowledge proofs for decision tree predictions and accuracy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2020, pp. 2039–2053.
- [4] J. Camenisch, M. Drijvers, A. Lehmann, G. Neven, and P. Towa, "Short threshold dynamic group signatures," in *Proc. Int. Conf. Security Cryptogr. Netw. (SCN)*, 2020, pp. 401–423.
- [5] D. Boneh, B. Bünz, and B. Fisch, "Batching techniques for accumulators with applications to IOPs and stateless blockchains," in *Proc. Annu. Int. Cryptol. Conf. (CRYPTO)*, 2019, pp. 561–586.
- [6] Z. Peng, C. Xu, H. Wang, J. Huang, J. Xu, and X. Chu, "P²B-trace: Privacy-preserving blockchain-based contact tracing to combat pandemics," in *Proc. Int. Conf. Manag. Data (SIGMOD)*, 2021, pp. 2389–2393.
- [7] P. Szalachowski, D. Reijnders, I. Homoliak, and S. Sun, "Strongchain: Transparent and collaborative proof-of-work consensus," in *Proc. USENIX Security Symp. (USENIX Security)*, 2019, pp. 819–836.
- [8] P. Gazi, A. Kiayias, and D. Zindros, "Proof-of-stake sidechains," in *Proc. IEEE Symp. Security Privacy (SP)*, 2019, pp. 139–156.
- [9] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [10] C. Xie, S. Koyejo, and I. Gupta, "Asynchronous federated optimization," 2019, *arXiv:1903.03934*.
- [11] P. Mohassel and Y. Zhang, "SecureML: A system for scalable privacy-preserving machine learning," in *Proc. IEEE Symp. Security Privacy (SP)*, 2017, pp. 19–38.
- [12] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, "BatchCrypt: Efficient homomorphic encryption for cross-silo federated learning," in *Proc. USENIX Conf. Usenix Annu. Tech. Conf. (USENIX ATC)*, 2020, pp. 493–506.
- [13] X. Xu, D. Zhu, X. Yang, S. Wang, L. Qi, and W. Dou, "Concurrent practical Byzantine fault tolerance for integration of blockchain and supply chain," *ACM Trans. Internet Technol.*, vol. 21, no. 1, pp. 1–17, 2021.
- [14] C. Dwork, "Differential privacy: A survey of results," in *Proc. Int. Conf. Theory Appl. Models Comput. (TAMC)*, 2008, pp. 1–19.
- [15] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Int. Conf. Artif. Intell. Stat. (AISTATS)*, 2017, pp. 1273–1282.
- [16] H. Fang and Q. Qian, "Privacy preserving machine learning with homomorphic encryption and federated learning," *Future Internet*, vol. 13, no. 4, p. 94, 2021.

- [17] R. Kanagavelu et al., "Two-phase multi-party computation enabled privacy-preserving federated learning," in *Proc. IEEE/ACM Int. Symp. Clust. Cloud Internet Comput. (CCGRID)*, 2020, pp. 410–419.
- [18] S. Zheng et al., "Asynchronous stochastic gradient descent with delay compensation," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 4120–4129.
- [19] X. Zhou, Y. Deng, H. Xia, S. Wu, and M. Bennis, "Resource allocation for time-triggered federated learning over wireless networks," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2022, pp. 2810–2815.
- [20] X. Zhang, X. Chen, J. K. Liu, and Y. Xiang, "DeepPAR and deepDPA: Privacy preserving and asynchronous deep learning for industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 16, no. 3, pp. 2081–2090, Mar. 2020.
- [21] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," 2016, *arXiv:1610.05492*.
- [22] N. Mehrabi, F. Morstatter, N. Saxena, K. Lerman, and A. Galstyan, "A survey on bias and fairness in machine learning," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–35, 2021.
- [23] C. Fung, C. J. M. Yoon, and I. Beschastnikh, "Mitigating sybils in federated learning poisoning," 2018, *arXiv:1808.04866*.
- [24] W.-M. Lee, *Beginning Ethereum Smart Contracts Programming, With Examples in Python, Solidity and JavaScript*. Berkeley, CA, USA: Apress, 2019.
- [25] Y. Lu, X. Huang, Y. Dai, S. Maharjan, and Y. Zhang, "Blockchain and federated learning for privacy-preserved data sharing in industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 16, no. 6, pp. 4177–4186, Jun. 2019.
- [26] S. Awan, F. Li, B. Luo, and M. Liu, "Poster: A reliable and accountable privacy-preserving federated learning framework using the blockchain," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2019, pp. 2561–2563.
- [27] Y. Liu, Y. Qu, C. Xu, Z. Hao, and B. Gu, "Blockchain-enabled asynchronous federated learning in edge computing," *Sensors*, vol. 21, no. 10, p. 3335, 2021.
- [28] Y. Lu, X. Huang, K. Zhang, S. Maharjan, and Y. Zhang, "Blockchain empowered asynchronous federated learning for secure data sharing in Internet of Vehicles," *IEEE Trans. Veh. Technol.*, vol. 69, no. 4, pp. 4298–4311, Apr. 2020.
- [29] L. Feng, Y. Zhao, S. Guo, X. Qiu, W. Li, and P. Yu, "BAFL: A blockchain-based asynchronous federated learning framework," *IEEE Trans. Comput.*, vol. 71, no. 5, pp. 1092–1103, May 2022.
- [30] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, nos. 3–4, pp. 211–407, 2014.
- [31] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Proc. Theory Cryptogr. Conf. (TCC)*, 2006, pp. 265–284.
- [32] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, pp. 1–19, 2019.
- [33] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.
- [34] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Int. Conf. Mach. Learn. Syst. (MLSys)*, vol. 2, 2020, pp. 429–450.



Xinru Yan received the B.E. degree from the Department of Automation and Software Engineering, Shanxi University, Taiyuan, China, in 2022. She is currently pursuing the M.E degree with the Department of Cyber Engineering, Xidian University, Xi'an, China.

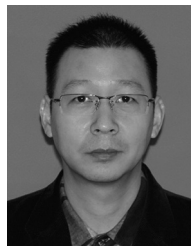
Her research interests include information security and applied cryptography.



Yinbin Miao (Member, IEEE) received the B.E. degree from the Department of Telecommunication Engineering, Jilin University, Changchun, China, in 2011, and the Ph.D. degree from the Department of Telecommunication Engineering, Xidian University, Xi'an, China, in 2016.

He is also a Postdoctoral Researcher with Nanyang Technological University, Singapore, from September 2018 to September 2019, and the City University of Hong Kong, Hong Kong, from December 2019 to December 2021. He is currently

an Associate Professor with the Department of Cyber Engineering, Xidian University. His research interests include information security and applied cryptography.



Xinghua Li received the M.E. and Ph.D. degrees in computer science from Xidian University, Xi'an, China, in 2004 and 2007, respectively.

He is currently a Professor with the School of Cyber Engineering, Xidian University. His research interests include wireless networks security, privacy protection, cloud computing, and security protocol formal methodology.



Kim-Kwang Raymond Choo (Senior Member, IEEE) received the Ph.D. degree in information security from Queensland University of Technology, Brisbane, QLD, Australia, in 2006.

He currently holds the Cloud Technology Endowed Professorship with The University of Texas at San Antonio, San Antonio, TX, USA.

Dr. Choo is the Founding Co-Editor-in-Chief of *ACM Distributed Ledger Technologies: Research and Practice*, Founding Chair of IEEE TEMS Technical Committee on Blockchain and Distributed

Ledger Technologies, an ACM Distinguished Speaker, and IEEE Computer Society Distinguished Visitor from 2021 to 2023, and a Web of Science's Highly Cited Researcher (Computer Science in 2021, Cross-Field in 2020). He is also the recipient of the 2019 IEEE Technical Committee on Scalable Computing Award for Excellence in Scalable Computing (Middle Career Researcher).

Xiangdong Meng, photograph and biography not available at the time of publication.



Robert H. Deng (Fellow, IEEE) received the B.E. degree from the National University of Defence Technology, Changsha, China, in 1981, and the M.S. and Ph.D. degrees from Illinois Institute of Technology, Chicago, IL, USA, in 1983 and 1985, respectively.

He has been an AXA Chair Professor of Cybersecurity and a Professor of Information Systems with the School of Information Systems, Singapore Management University, Singapore, since 2004. His research interests include data security and

privacy, multimedia security, network, and system security.

Dr. Deng has received the Distinguished Paper Award for NDSS in 2012, the Best Paper Award for *Computational Management Science* in 2012, and the Best Journal Paper Award for IEEE Communications Society in 2017. He served/is serving on the editorial boards of many international journals, including IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY and IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING.